



UNIVERSITÉ DE
MONTPELLIER

Faculté des sciences de Montpellier

Année Universitaire 2025-2026

Rapport de projet

Analyse Numérique 3

Rédigé par :

Adrien **Martinelli** et

Benjamin **Bronstein**

18 avril 2026

Exercice 1

L'équation de Black-Scholes s'écrit :

$$-\frac{1}{2}\sigma^2 S^2 V''(S) - rSV'(S) + rV(S) = f(S), \quad S \in (0, S_{\max})$$

Question 1

On pose $u(x) = V(e^x)$.

On calcule $u'(x) = e^x V'(e^x)$ et $u''(x) = e^x V'(e^x) + e^{2x} V''(e^x) = u'(x) + e^{2x} V''(e^x)$
En isolant V'' dans l'équation précédente nous obtenons :

$$V''(e^x) = e^{-2x} (u'(x) + u''(x))$$

Puis en injectant dans l'équation Black Scholes de départ, nous obtenons :

$$-\frac{1}{2}\sigma^2 u''(x) + \left(\frac{1}{2}\sigma^2 - r\right)u'(x) + ru(x) = f(e^x), \quad S \in (-\infty, \ln(S_{\max}))$$

Avec les conditions suivante :

$$\begin{cases} \lim_{x \rightarrow -\infty} u(x) = 0 \\ u(\ln(S_{\max})) = S_{\max} - K \end{cases}$$

Question 2

Les conditions aux limites n'étant pas homogènes, nous allons faire un relèvement.

Soit $u_g \in H^1(\Omega)$ telle que $u_g(\ln(S_{\max})) = S_{\max} - K$ et $\lim_{x \rightarrow -\infty} u_g(x) = 0$.

On a $(u - u_g)|_{\partial\Omega} = 0$ En posant $\tilde{u} = u - u_g$, on écrit alors :

$$u(x) = \tilde{u}(x) + u_g(x)$$

Ainsi on a bien $\tilde{u} \in H_0^1(\Omega)$, on peut donc calculer la forme variationnelle en prenant $v \in H_0^1(\Omega)$. On pose $\ln(S_{\max}) := S$

$$\int_{-\infty}^S -\frac{1}{2}\sigma^2 (\tilde{u} + u_g)'' v + \left(\frac{1}{2}\sigma^2 - r\right)(\tilde{u} + u_g)' v + r(\tilde{u} + u_g)v = \int_{-\infty}^S f(e^x)v$$

En intégrant par parties :

$$-\frac{1}{2}\sigma^2 \int_{-\infty}^S (\tilde{u} + u_g)'' v + \int_{-\infty}^S \left(\frac{1}{2}\sigma^2 - r\right) (\tilde{u} + u_g)' v + \int_{-\infty}^S r(\tilde{u} + u_g) v = \int_{-\infty}^S f(e^x) v$$

Qui donne :

$$\begin{aligned} & -\frac{1}{2}\sigma^2 \left[(\tilde{u} + u_g)' v \right]_{-\infty}^S + \frac{1}{2}\sigma^2 \int_{-\infty}^S (\tilde{u} + u_g)' v' \\ & + \int_{-\infty}^S \left(\frac{1}{2}\sigma^2 - r\right) (\tilde{u} + u_g)' v \\ & + \int_{-\infty}^S r(\tilde{u} + u_g) v \\ & = \int_{-\infty}^S f(e^x) v \end{aligned}$$

On obtient donc :

$$\begin{aligned} & \int_{-\infty}^S \frac{1}{2}\sigma^2 \tilde{u}' v' + \left(r - \frac{1}{2}\sigma^2\right) \tilde{u} v' + r \tilde{u} v \, dx \\ & = \int_{-\infty}^S f(e^x) v + \left(\frac{1}{2}\sigma^2 - r\right) u_g v' - r u_g v - \frac{1}{2}\sigma^2 u_g' v' \, dx \end{aligned}$$

On explicite l'application bilinéaire et linéaire :

$$\begin{cases} a(\tilde{u}, v) = \int_{-\infty}^S \frac{1}{2}\sigma^2 \tilde{u}' v' + \left(r - \frac{1}{2}\sigma^2\right) \tilde{u} v' + r \tilde{u} v \, dx \\ l(v) = \int_{-\infty}^S f(e^x) v + \left(\frac{1}{2}\sigma^2 - r\right) u_g v' - r u_g v - \frac{1}{2}\sigma^2 u_g' v' \, dx \end{cases}$$

Question 3

Nous allons choisir un second membre pour appliquer des méthodes de résolution à l'équation.

D'après l'énoncé, la solution exacte est donnée par $V(S) = S^2$, $\forall S \in [0, 200]$ Ce qui donne, en utilisant cette solution exacte dans l'équation :

$$-\frac{1}{2}\sigma^2 S^2 \times 2 - r 2S^2 + r S^2 = -(\sigma^2 + r) S^2 = f(S)$$

Et dans le cadre de l'équation que nous étudions, $f(e^x) = -(\sigma^2 + r)e^{2x}$

Dans le cadre de cette étude, nous allons adapter les conditions au bord en imposant

$$\begin{cases} u(x) = 0 & \text{et} \\ x \rightarrow -\infty \\ u(\ln(S_{max})) = 40000 \end{cases}$$

Nous faisons le choix de cette condition au bord droit car la solution exacte évaluée à ce bord vaut $V(e^{\ln(S_{max})}) = V(S_{max}) = S_{max}^2 = 200^2 = 40000$

On va maintenant implémenter la résolution en FreeFem++ afin d'obtenir un profil pour la solution numérique et d'obtenir l'ordre de convergence en éléments finis P1 et P2.

Voici une première résolution en éléments P1, l'intervalle de résolution est $]-100, \ln(S_{max})[$ pour 50 mailles.



FIGURE 1 – Solution numérique Black-Scholes, éléments finis P1

Ci-dessous une deuxième résolution numérique, avec le même nombre de mailles et le même intervalle mais en éléments finis P2 :



FIGURE 2 – Solution numérique Black-Scholes, éléments finis P2

Voici le code menant à la résolution P1, la fin du code nous récupérons les valeurs de la solution pour l'afficher dans un graphique à l'aide d'un code python.

```

1  real sigma = 0.2;
2  real r = 0.05;
3  real K = 50;
4
5  int Nint = 100;
6  real Smax = 200;
7  real xmin = log(0.001);
8  real xmax = log(Smax);
9
10 mesh Th = square(Nint,1,[ xmin + (xmax - xmin)*x , y/2 ]);
11
12 func f = -(sigma^2 + r)*exp(2*x);
13
14 fespace Vh(Th,P1);
15 Vh uh,vh;
16
17 problem bs(uh,vh,solver=LU)=
18   int2d(Th)(0.5*sigma^2*dx(uh)*dx(vh) + (r - 0.5*sigma^2)*dx(uh)*vh +
19     r*uh*vh)
20   -int2d(Th)(f*vh)
21   + on(4,uh = 0)
22   + on(2,uh = 40000);
23
24 bs;
25
26 plot(uh,wait=1,cmm="Solution Black-Scholes",fill=true);

```

```

27 int Nplot = 100;
28
29 ofstream file("solution.txt");
30
31 for (int i = 0; i <= Nplot; i++) {
32     real xx = xmin + (xmax - xmin)*i/Nplot;
33     file << xx << " " << uh(xx,0.5) << endl;
34 }

```

Listing 1 – Resolution P1

La solution tracée donne :

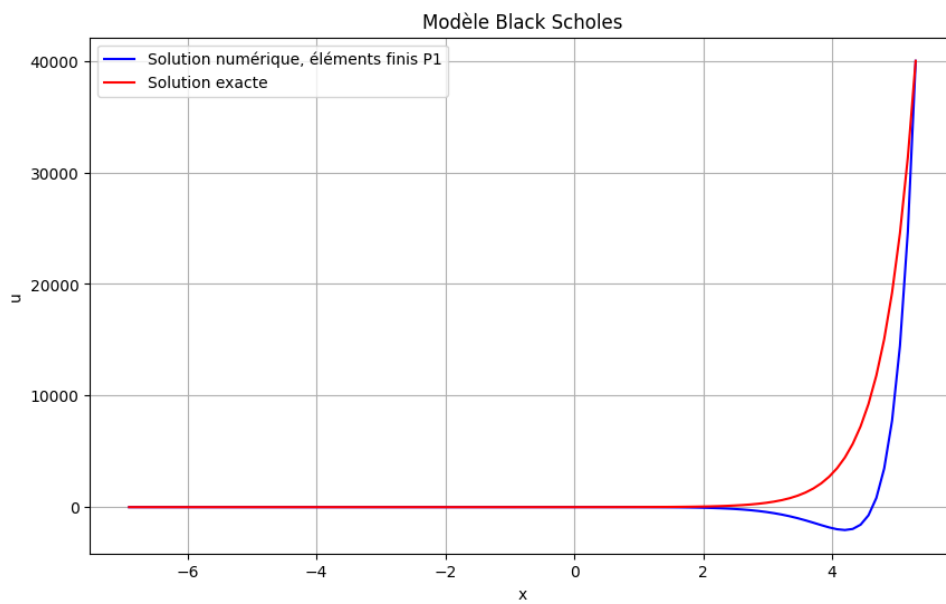


FIGURE 3 – Solution numérique Black-Scholes, éléments finis P1

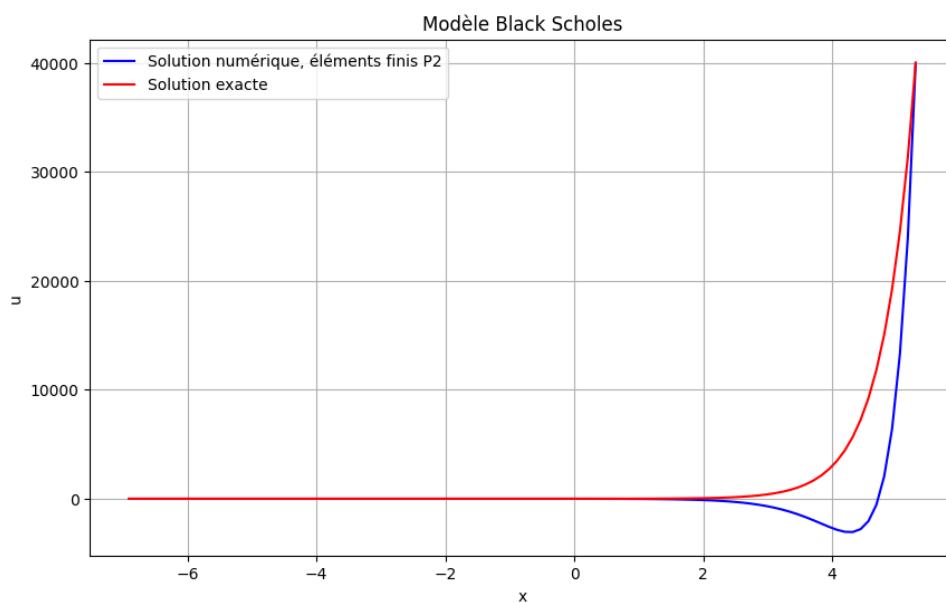


FIGURE 4 – Solution numérique Black-Scholes, éléments finis P2

Nous avons pris $u_{exact}(x) = e^{2x}$ pour la solution exacte, puisque $V(S) = S^2$ est la solution exacte du modèle de base.

Après avoir crée une boucle de convergence en norme L^2 et H^1 pour les éléments finis $P1$ et $P2$, nous obtenons les graphiques de convergence suivants :

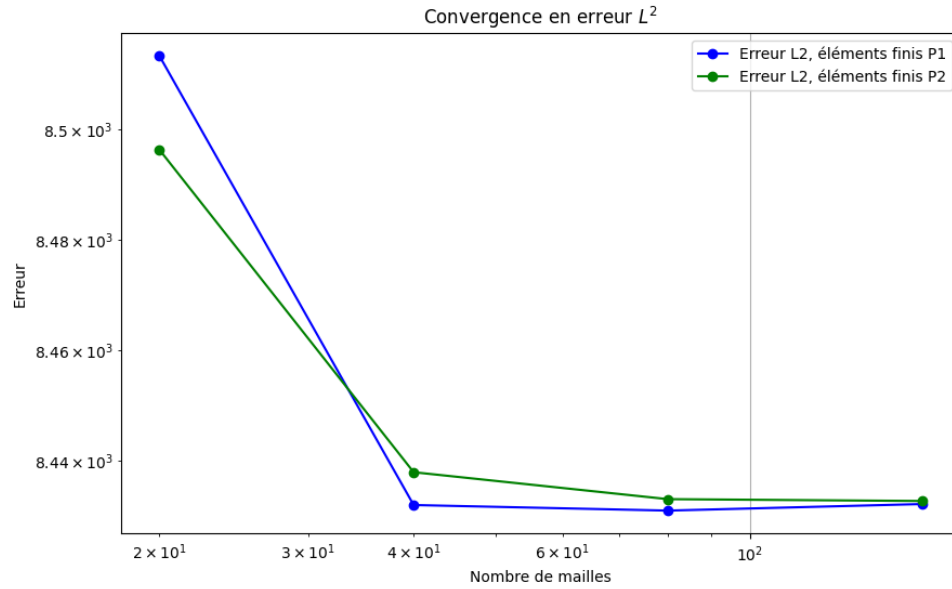


FIGURE 5 – Convergence en norme L^2

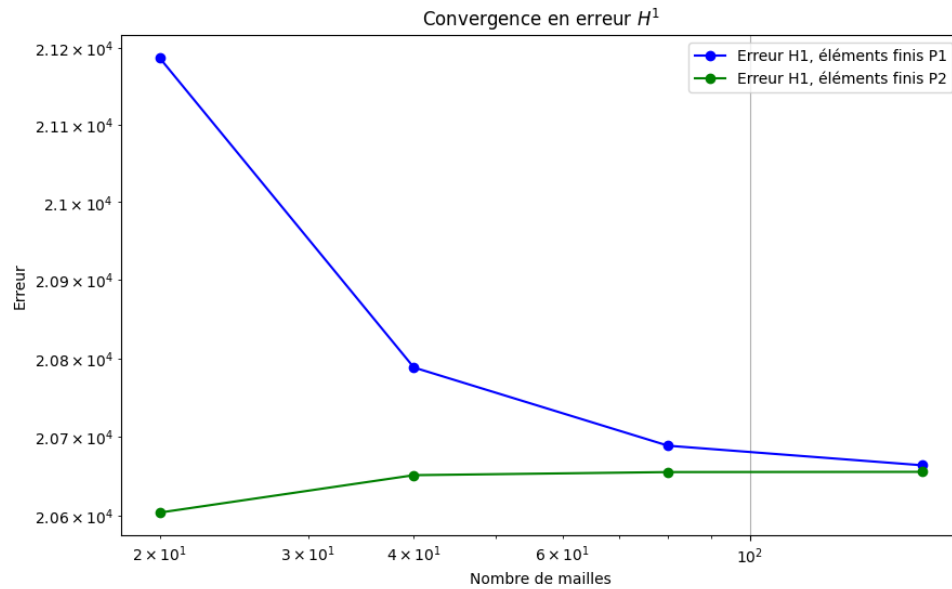


FIGURE 6 – Convergence en norme H^1

```

1  int Nb = 10;
2  int n = 5;
3  real Smax = 200;
4  real xmin = log(0.001);
5  real xmax = log(Smax);
6  real sigma = 0.2;

```

```

7  real r = 0.05;
8
9  func w = exp(2*x);
10 func f = -(sigma^2 + r)*exp(2*x);
11
12 ofstream erreurP1("erreur_P1.txt");
13 ofstream erreurP2("erreur_P2.txt");
14
15
16 for (int k = 1; k < n; k++) {
17
18     int N = Nb*2^k;
19
20     mesh Th = square(N,1,[ xmin + (xmax - xmin)*x , y/2 ]);
21     fespace Vh(Th, P1);
22     Vh uh, vh;
23
24     problem bs(uh,vh,solver=LU)=
25     int2d(Th)(0.5*sigma^2*dx(uh)*dx(vh) + (r - 0.5*sigma^2)*dx(uh)*vh +
26             r*uh*vh)
27     -int2d(Th)(f*vh)
28     + on(4,uh = 0)
29     + on(2,uh = 40000);
30
31     bs;
32
33     real errL2 = sqrt(int2d(Th)((uh - w)^2));
34     real errH1 = sqrt(int2d(Th)((uh - w)^2 + (dx(uh) - 2*w)^2));
35
36     erreurP1 << N << " " << errL2 << " " << errH1 << endl;
37 }
38
39 for (int k = 1; k < n; k++) {
40
41     int N = Nb*2^k;
42
43     mesh Th = square(N,1,[ xmin + (xmax - xmin)*x , y/2 ]);
44     fespace Vh(Th, P2);
45     Vh uh, vh;
46
47     problem bs(uh,vh,solver=LU)=
48     int2d(Th)(0.5*sigma^2*dx(uh)*dx(vh) + (r - 0.5*sigma^2)*dx(uh)*vh +
49             r*uh*vh)
50     -int2d(Th)(f*vh)
51     + on(4,uh = 0)
52     + on(2,uh = 40000);
53
54     bs;
55
56     real errL2 = sqrt(int2d(Th)((uh - w)^2));
57     real errH1 = sqrt(int2d(Th)((uh - w)^2 + (dx(uh) - 2*w)^2));
58
59     erreurP2 << N << " " << errL2 << " " << errH1 << endl;
60 }

```

Listing 2 – Calcul de l'erreur

Les courbes de convergence et les graphiques affichant la solution contre la solution

exacte ne semblent pas exactement coïncider. Nous faisons face à une incohérence dans les conditions au bord. Au départ nous imposons $V(S_{max}) = S_{max} - K$, donc en prenant $S_{max} = 200$ et $K = 50$ nous avons $V(S_{max}) = 150$. Mais avec la solution exacte $V(S) = S^2$ nous avons $S_{max}^2 = S_{max} - K$, ce qui donnerait $40000 = 150$, ce qui n'est pas possible. Nous faisons donc le choix d'imposer $u(\ln(S_{max})) = V(S_{max}) = 40000$ au bord, ce qui n'est peut être pas cohérent avec le contexte du modèle choisi.

Question 4

Nous allons maintenant étudier l'influence des paramètres sur le modèle.

Nous allons prendre $\sigma = 0.1$ pour simuler un marché peu volatile et reprendre les courbes précédentes :

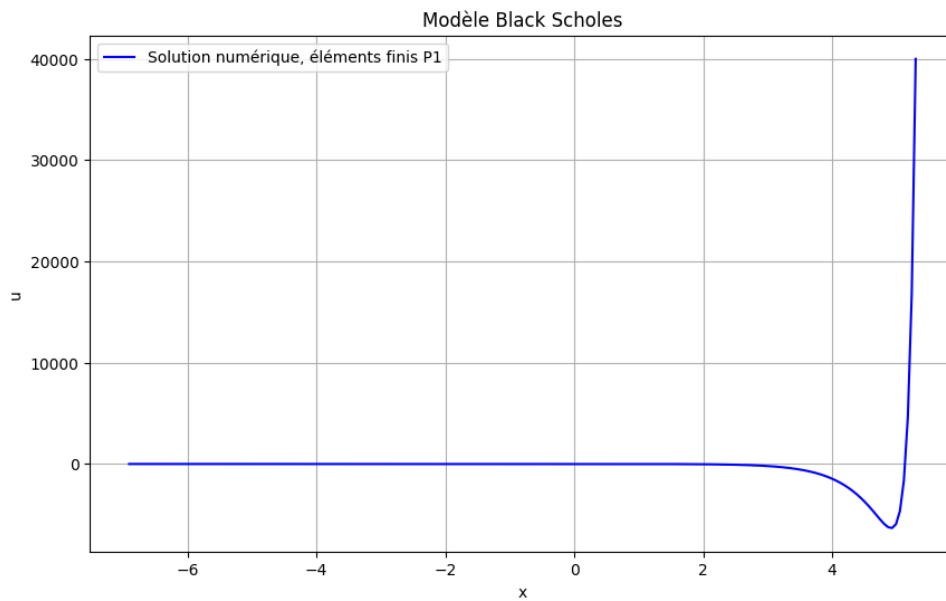


FIGURE 7 – $\sigma = 0.1$, $P1$



FIGURE 8 – $\sigma = 0.1$, $P1$

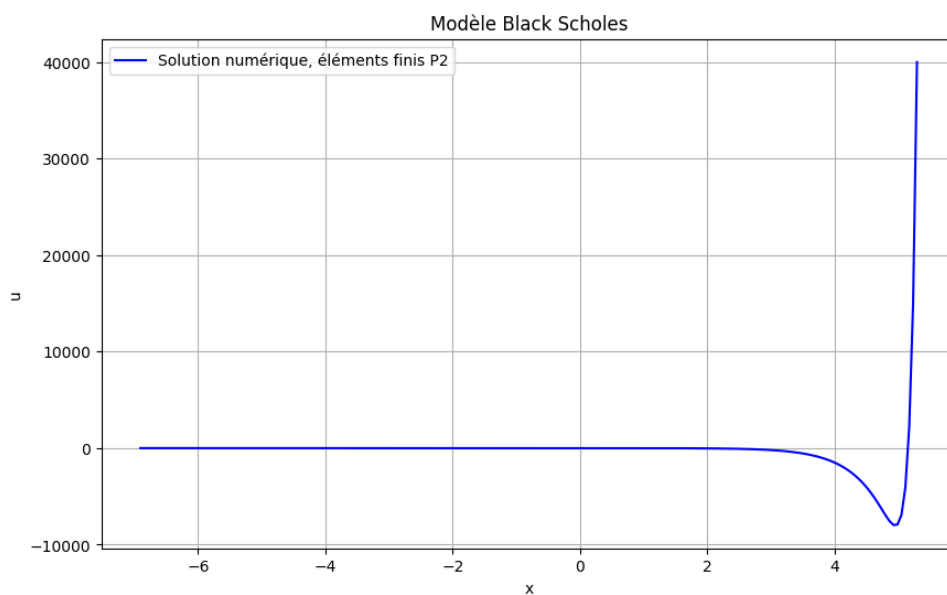


FIGURE 9 – $\sigma = 0.1$, $P2$

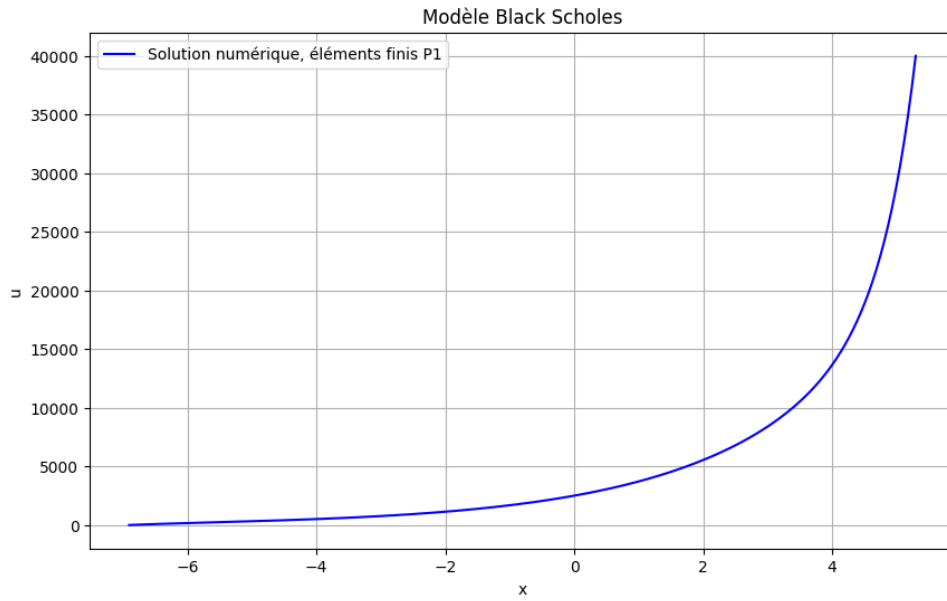
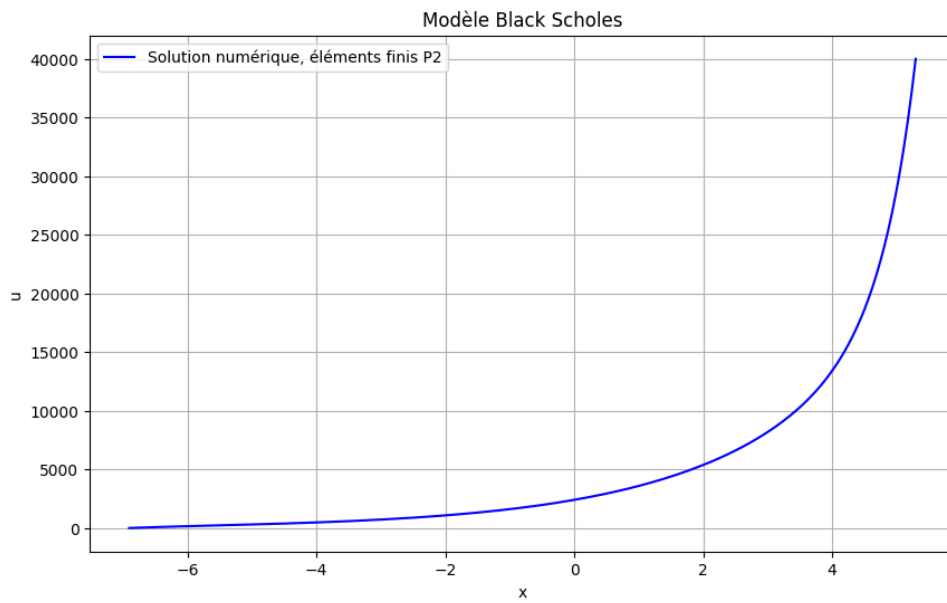


FIGURE 10 – $\sigma = 0.1$, $P2$

Maintenant nous choisissons $\sigma = 0.5$. En conservant les mêmes paramètres que précédemment, les solutions numériques en $P1$ et $P2$ donnent :



FIGURE 11 – $\sigma = 0.5$, $P1$

FIGURE 12 – $\sigma = 0.5$, $P1$ FIGURE 13 – $\sigma = 0.5$, $P2$ FIGURE 14 – $\sigma = 0.5$, $P2$

Nous observons que le paramètre σ semble avoir un effet régularisant sur la solution, lissant le profil de la solution obtenue.

Question 5

Nous reprenons $\sigma = 0.2$ et choisissons $r = 0.1$, nous affichons les solutions en $P1$ et $P2$:



FIGURE 15 – $r = 0.1$, $P1$

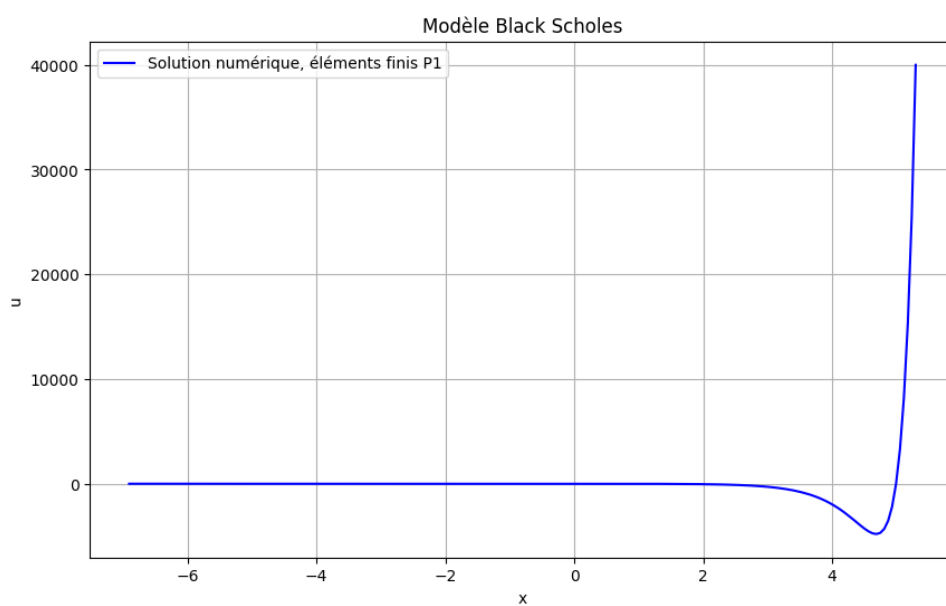
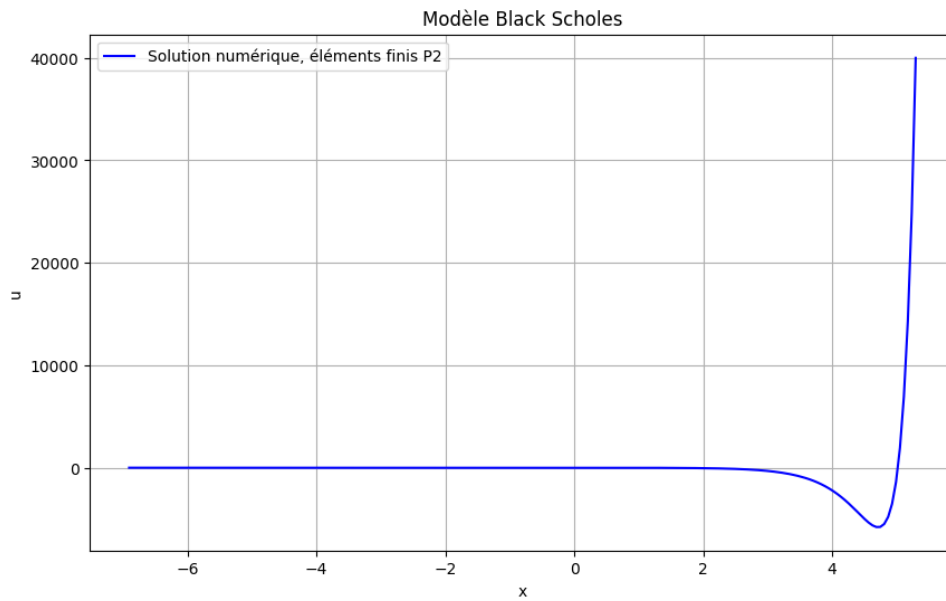


FIGURE 16 – $r = 0.1$, $P1$



FIGURE 17 – $r = 0.1$, $P2$

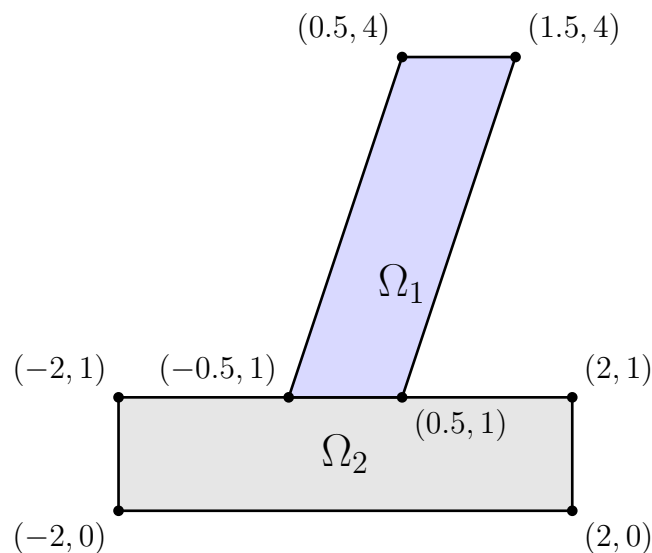
FIGURE 18 – $r = 0.1$, $P2$

Le paramètre r semble accentuer la pente au voisinage du bord droit.

Exercice 2

Question 1

On veut modéliser un pied d'atterrissage composé d'une semelle rectangulaire et d'une jambe inclinée. On décide que les dimensions du pieds seront comme représentées sur ce dessin :



On a donc le code suivant :

```
1 real xMinBase = -2.0, xMaxBase = 2.0;
2 real yBase = 0.0, hBase = 1.0;
```

```

3
4 real xminParBas = -0.5, xmaxParBas = 0.5;
5 real hPar = 4.0;
6 real xminParHaut = 0.5, xmaxParHaut = 1.5;
7
8 int n = 5;
9
10 border bas(t=xMinBase, xMaxBase) {x=t; y=yBase; label=1;}
11 border droiteBase(t=yBase, hBase) {x=xMaxBase; y=t; label=3;}
12 border hautBaseD(t=xMaxBase, xMaxParBas) {x=t; y=hBase; label=3;}
13 border droitePar(t=0, 1) {x=xMaxParBas + t*(xMaxParHaut - xMaxParBas); y
    =hBase + t*(hPar - hBase); label=3;}
14 border hautPar(t=xMaxParHaut, xMinParHaut) {x=t; y=hPar; label=2;} //
    Gamma_haut
15 border gauchePar(t=1, 0) {x=xMinParBas + t*(xMinParHaut - xMinParBas); y
    =hBase + t*(hPar - hBase); label=3;}
16 border hautBaseG(t=xMinParBas, xMinBase) {x=t; y=hBase; label=3;}
17 border gaucheBase(t=hBase, yBase) {x=xMinBase; y=t; label=3;}
18
19 mesh Th = buildmesh(bas(4*n) + droiteBase(n) + hautBaseD(2*n) +
    droitePar(3*n) + hautPar(n) + gauchePar(3*n) + hautBaseG(2*n) +
    gaucheBase(n));
20
21 plot(Th, wait=true, cmm="Maillage initial du pied d'atterrissage");

```

Qui nous donne le maillage suivant :

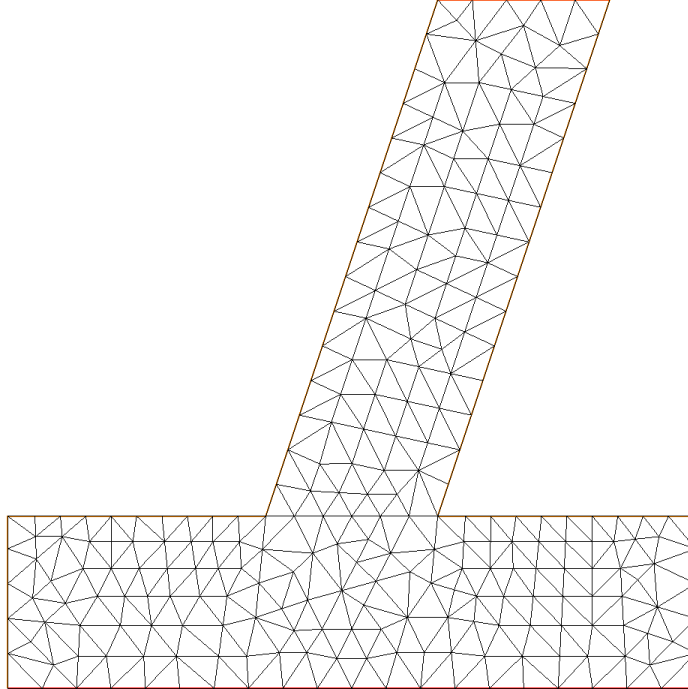


FIGURE 19 – Maillage Train

On s'attaque maintenant à la modélisation du problème d'élasticité sur $\Omega = \Omega_1 \cap \Omega_2$. La modélisation se fait dans le contexte suivant :

— Propriétés du matériau :

- $\mu = 27.0$: Second coefficient de Lamé.
- $\lambda = 52.0$: Premier coefficient de Lamé.
- Forces volumiques :
 - $g = 3.71 \text{ m/s}^2$: Accélération de la pesanteur (correspondant à la gravité martienne).
 - $\rho = 2700.0 \text{ kg/m}^3$: Masse volumique du matériau
 - Le vecteur force volumique est $\mathbf{f} = (f_x, f_y) = (0, -\rho g)$.
- Forces surfaciques :
 - $p = 1.4 \times 10^4 \text{ Pa}$: Pression descendante appliquée sur la frontière haute.

Il peut être intéressant de remarquer que d'après les propriétés physiques du matériau, le train est probablement constitué principalement d'aluminium.

Le tenseur des déformations est défini par :

$$\varepsilon(\mathbf{u}) = \frac{\nabla \mathbf{u} + (\nabla \mathbf{u})^t}{2}$$

Le tenseur des contraintes s'écrit :

$$\sigma_{i,j}(\mathbf{u}) = \lambda \delta_{i,j} \text{tr}(\varepsilon) + 2\mu \varepsilon_{i,j}, \quad 1 \leq i, j \leq 2$$

où μ et λ sont les coefficients de Lamé du matériau.

La formulation variationnelle de notre problème est la suivante : trouver $u \in V$ tel que pour toute fonction test $v \in V$ (nulle sur Γ_{sol}) on ait :

$$\int_{\Omega} (2\mu \varepsilon(u) : \varepsilon(v) + \lambda \text{div}(u) \text{div}(v)) d\Omega = \int_{\Omega} \mathbf{f} \cdot \mathbf{v} d\Omega + \int_{\Gamma_{haut}} F_{surf} \cdot \mathbf{v} ds$$

Avec :

- $\mathbf{f} = (0, -\rho g)$ la force volumique de gravité.
- $F_{surf} = (0, -p)$ la pression appliquée sur le bord supérieur.

On obtient donc le code suivant :

```

1  real mu = 27.0;
2  real lambda = 52.0;
3  real g = 3.71;
4  real p = 1.4e4;
5
6  real fx = 0.0;
7  real fy = -2700.0 * g;
8
9  //On suppose que on cree le maillage ici
10
11 fespace Vh(Th, [P2, P2]);
12 Vh [u1, u2], [v1, v2];
13
14 macro div(u1, u2) (dx(u1) + dy(u2)) //
15
16 macro epsilonSquare(u1, u2, v1, v2) (
17   dx(u1)*dx(v1) + dy(u2)*dy(v2) + 0.5*(dy(u1) + dx(u2))*(dy(v1) + dx(
18   v2))
19 ) //
```

```

19
20
21 problem Elasticite([u1, u2], [v1, v2]) =
22     int2d(Th)(
23         lambda * div(u1, u2) * div(v1, v2)
24         + 2.0 * mu * epsilonSquare(u1, u2, v1, v2)
25     )
26     - int2d(Th)( fx*v1 + fy*v2 )
27     - int1d(Th, 2)( -p * v2 )
28     + on(1, u1=0, u2=0);
29
30
31
32 Elasticite;
33
34 plot([u1, u2], value=true, wait=true, cmm="Champ de déplacement [u1, u2]
    ");

```

On obtient le résultat suivant :

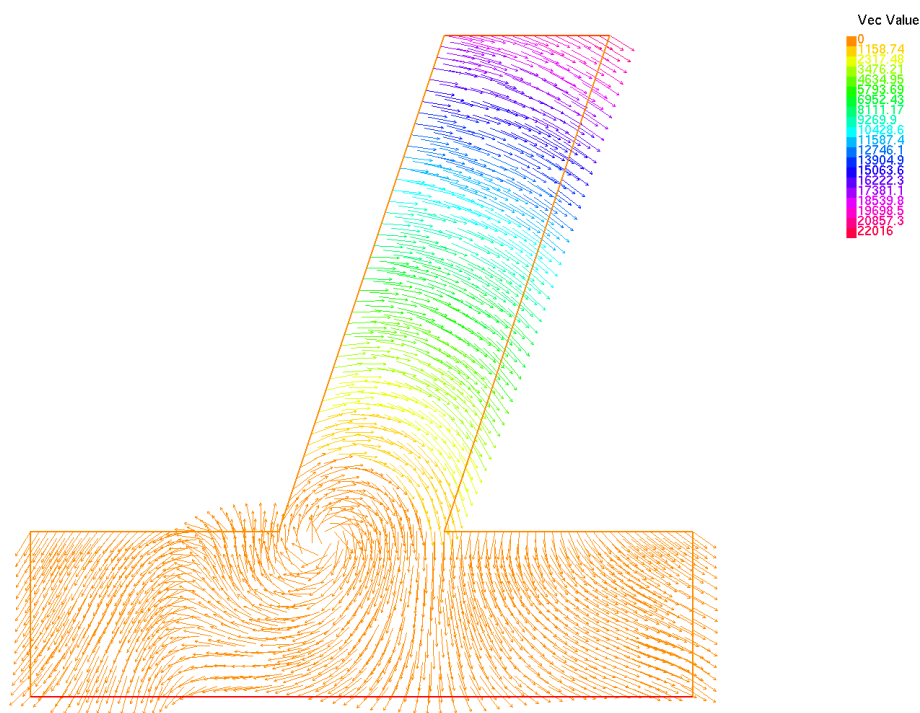


FIGURE 20 – Effort Train

On remarque ici que les efforts semblent disproportionnés. Cela vient des coefficients de Lamé. En effet, comme indiqué plus haut nous avons choisis $\mu = 27.0$ et $\lambda = 52.0$. Dans le contexte du problème, le train est probablement en aluminium, et ces coefficients s'expriment en réalité en GigaPascals.

En modifiant les coefficients et en les adaptant, on peut trouver la déformation de la Figure 21 qui est plus plausible.

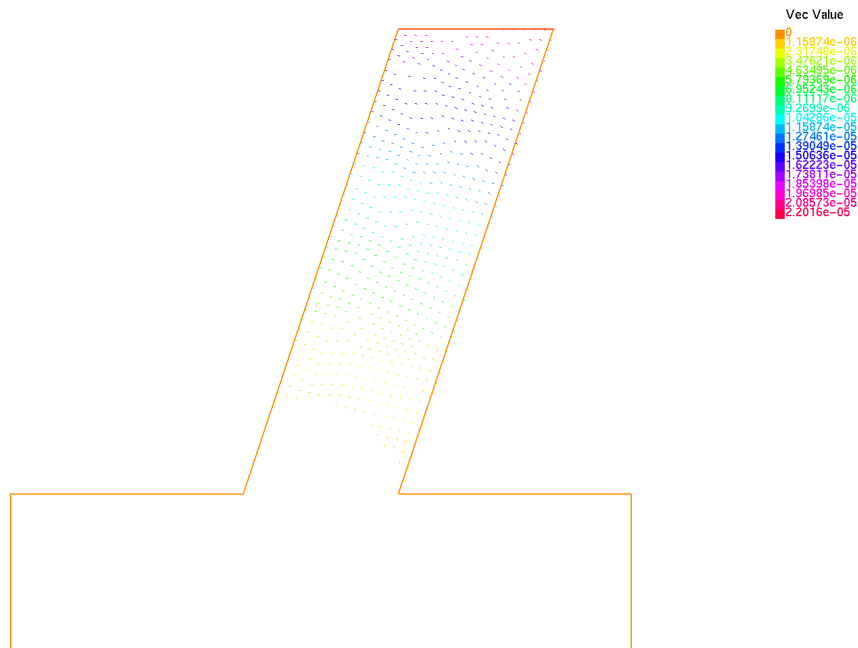


FIGURE 21 – Effort Train nouveaux coefficients

En amplifiant par 10000 la déformation, on peut la visualiser sur le maillage comme dans la Figure 33

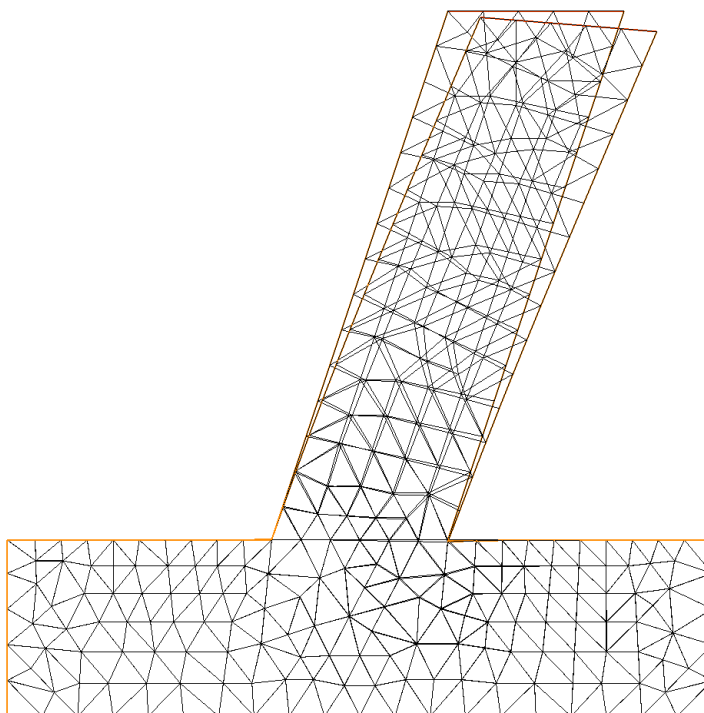


FIGURE 22 – Déformation Train nouveaux coefficients

Question 2

Ici, on considère la nature meuble du sol martien. La condition sur le bord inférieur Γ_{sol} est remplacée et on traduit la réaction normale du sol martien lorsque le pied de

l'atterrisseur s'enfonce par :

$$\sigma(u)n = -ku_2n$$

où $k = 10^7 \text{ N} \cdot \text{m}^{-3}$.

On a donc le code suivant :

```

1 problem Elasticite2([u1, u2], [v1, v2]) =
2   int2d(Th)(
3     lambda * div(u1, u2) * div(v1, v2)
4     + 2.0 * mu * epsilonSquare(u1, u2, v1, v2)
5   )
6   + int1d(Th, 1)( k * u2 * v2 )
7
8   - int2d(Th)( fx*v1 + fy*v2 )
9   - int1d(Th, 2)( -p * v2 )
10
11   + on(1, u1=0);
12
13 Elasticite2;
```

Qui donne la figure suivante :

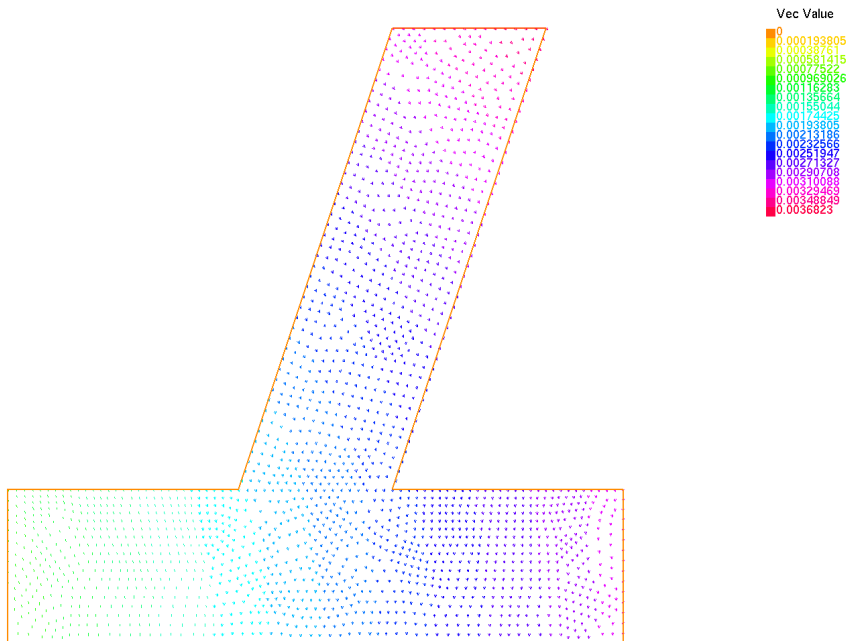


FIGURE 23 – Déplacement matériel de l'atterrisseur sur un sol élastique.

La Figure 23 illustre le champ de déplacement de la structure. Contrairement au modèle précédent, on observe que la base de l'atterrisseur ne reste pas fixe ($u \neq 0$ sur Γ_{sol}) et qu'elle s'enfonce.

Avec les coefficients de Lamé de l'énoncé, on obtiendrait plutôt la Figure 24.

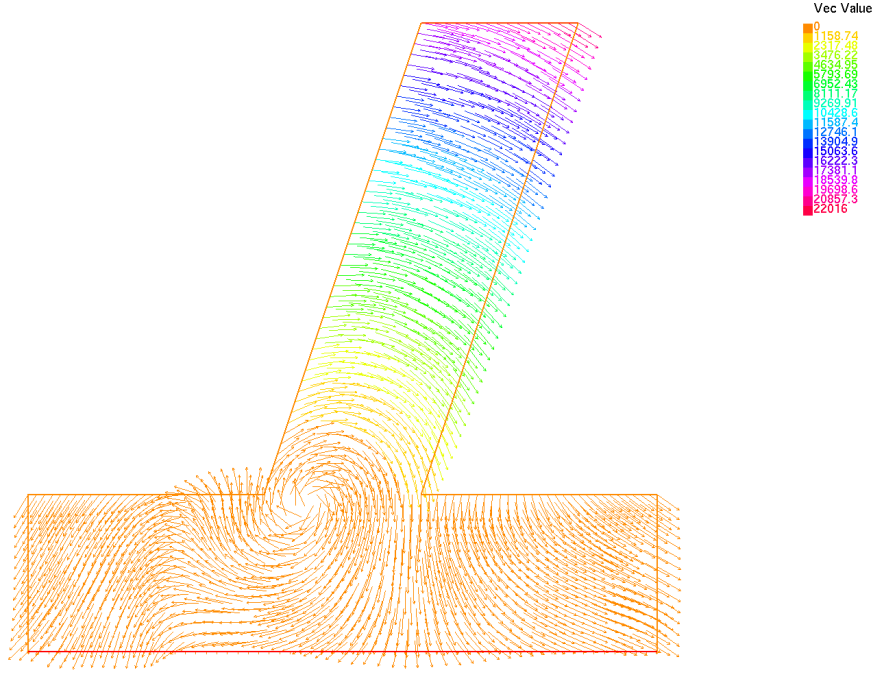


FIGURE 24 – Déplacement matériel de l'atterrisseur sur un sol élastique 2.

Question 3

On va à présent tracer la contrainte de Von Mises qui permet de visualiser où le matériau risque de céder :

$$\sigma_{vm} = \sqrt{\sigma_{11}^2 - \sigma_{11}\sigma_{22} + \sigma_{22}^2 + 3\sigma_{12}^2}$$

Le tenseur des déformations linéarisé s'exprime en dimension 2 sous la forme matricielle :

$$\epsilon(\mathbf{u}) = \begin{pmatrix} \frac{\partial u_1}{\partial x} & \frac{1}{2} \left(\frac{\partial u_1}{\partial y} + \frac{\partial u_2}{\partial x} \right) \\ \frac{1}{2} \left(\frac{\partial u_2}{\partial x} + \frac{\partial u_1}{\partial y} \right) & \frac{\partial u_2}{\partial y} \end{pmatrix}$$

On obtient donc :

$$\text{Tr}(\epsilon) = \epsilon_{11} + \epsilon_{22} = \frac{\partial u_1}{\partial x} + \frac{\partial u_2}{\partial y} = \text{div}(\mathbf{u})$$

En réutilisant la formule :

$$\sigma_{i,j}(\mathbf{u}) = \lambda \delta_{i,j} \text{tr}(\epsilon) + 2\mu \epsilon_{i,j}, \quad 1 \leq i, j \leq 2$$

On obtient les coefficients de la matrice suivants :

$$\begin{aligned} \sigma_{11} &= \lambda \text{div}(u) + 2\mu \frac{\partial u_1}{\partial x} \\ \sigma_{22} &= \lambda \text{div}(u) + 2\mu \frac{\partial u_2}{\partial y} \\ \sigma_{12} &= 2\mu \epsilon_{12} = \mu \left(\frac{\partial u_1}{\partial y} + \frac{\partial u_2}{\partial x} \right) \end{aligned}$$

On l'applique au code suivant :

```

1 macro sigma11(u1, u2) (lambda * div(u1, u2) + 2.0 * mu * dx(u1)) //
2 macro sigma22(u1, u2) (lambda * div(u1, u2) + 2.0 * mu * dy(u2)) //
3 macro sigma12(u1, u2) (mu * (dy(u1) + dx(u2))) //
4
5 fespace VhP1(Th, P1);
6 VhP1 vonMises;
7
8 vonMises = sqrt( sigma11(u1,u2)^2 - sigma11(u1,u2)*sigma22(u1,u2) +
9                 sigma22(u1,u2)^2 + 3.0*sigma12(u1,u2)^2 );
10 plot(vonMises, value=true, fill=true, wait=true, cmm="Contrainte de Von
    Mises (Zones de danger)");

```

On obtient donc finalement la Figure 25, où on a mis $\mu = 27.0$ et $\lambda = 52.0$.

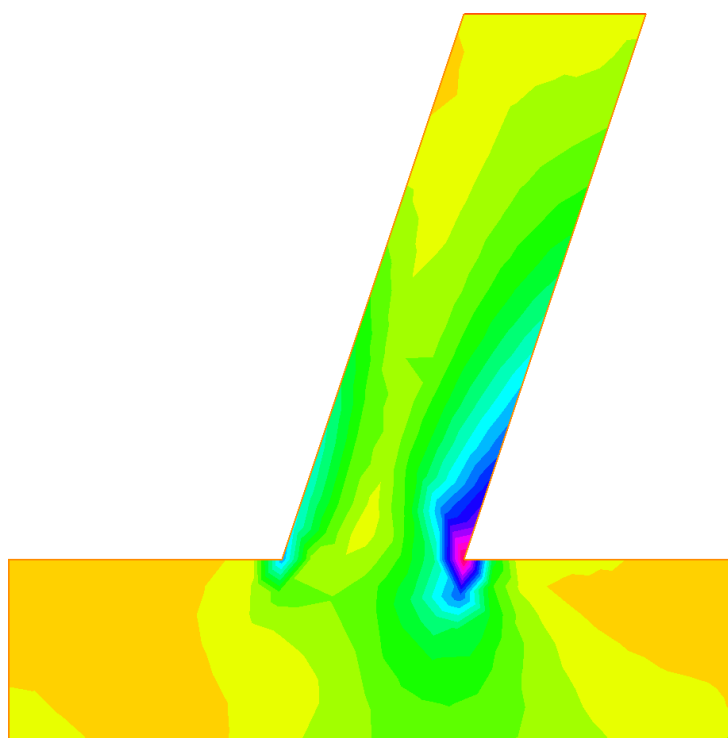


FIGURE 25 – Déplacement matériel de l'atterrisseur sur un sol élastique.

Le maximum d'effort (la contrainte de Von Mises la plus élevée) se constate au niveau du coin rentrant où la jambe inclinée rejoint le haut de la semelle rectangulaire.

Exercice 3

On utilise le paramétrage fourni dans l'énoncé et on utilise un domaine centré sur l'aile correspondant à un cercle :

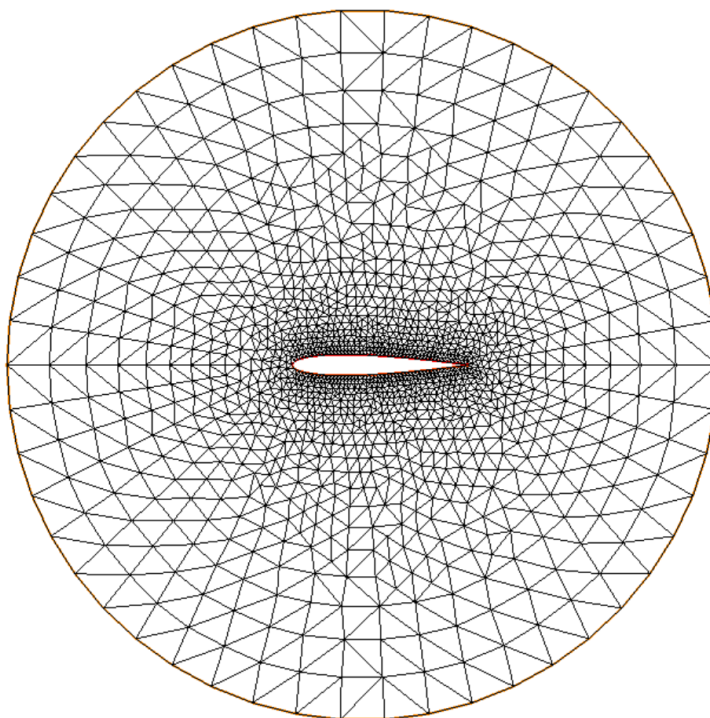


FIGURE 26 – Maillage du domaine autour de l'aile

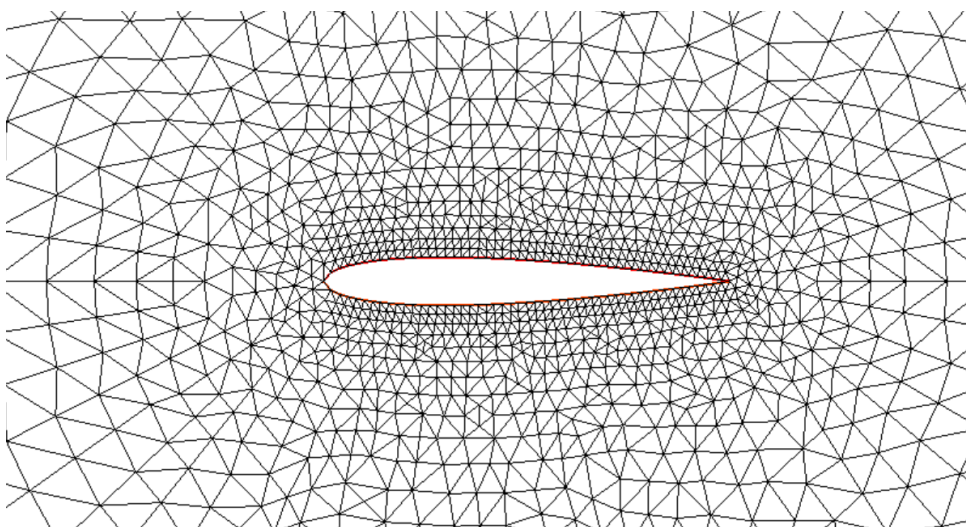


FIGURE 27 – Maillage autour de l'aile

```

1  int n = 50;
2
3  border C1(t=0,1){x=t; y= 0.17735*sqrt(t) - 0.075597*t - 0.212836*t^2 +
    0.17363*t^3 - 0.06254*t^4; label=1;}
4  border C2(t=0,1){x=1-t; y=-(0.17735*sqrt(1 - t) - 0.075597*(1-t) -
    0.212836*(1-t)^2 + 0.17363*(1-t)^3 - 0.06254*(1-t)^4); label=2;}
5  border cercle(t=0,1){x= 0.4 + 2*cos(2*pi*t); y=2*sin(2*pi*t); label=3;}
6
7
8  mesh Th = buildmesh(cercle(n) + C1(n) + C2(n));

```

Listing 3 – Définition du maillage

On impose que la vitesse du fluide soit nulle au bord de l'aile, soit $\partial_n \phi = \nabla \phi \cdot n = 0$ (no slip condition). Au bord du domaine entourant l'aile, on considère la vitesse du fluide connu, de sorte qu'on ait $\partial_n \phi = u_\infty$, pour un certain u_∞ .

Dans le cas d'un fluide non visqueux et irrotationnel, la vitesse dérive d'un potentiel ϕ . On considère l'équation :

$$\operatorname{div}(\rho \nabla \phi) = 0$$

Où la densité est donnée par $\rho = \left(1 - \frac{\gamma-1}{\gamma+1} \frac{|\nabla \phi|^2}{c^{\star 2}}\right)^{\frac{1}{\gamma}-1}$

On prend ici $\gamma = \frac{7}{5}$ et $c^\star = 1$ Donc la densité est donnée par la forme simplifiée :

$$\rho = \left(1 - \frac{1}{6} |\nabla \phi|^2\right)^{-\frac{2}{7}}$$

Pour résoudre le problème avec conditions au bord, nous allons utiliser une méthode de point fixe en résolvant un problème linéaire à l'aide d'éléments finis $P1$.

On se fixe une densité initiale $\rho_0(x, y) > 0$, que nous prendrons constante égale à 1. En ensuite pour les itérations de la méthode de point fixe, nous allons considérer le problème linéaire

$$\begin{cases} \operatorname{div}(\rho_{n-1} \nabla \phi_n) = 0, & \text{dans } \Omega \\ \partial_n \phi_n = 0 & \text{sur } \partial\Omega \\ \partial_n \phi_n = u_\infty \cdot n & \text{sur la frontière extérieure} \end{cases}$$

Nous établissons la formulation variationnelle associée à ce problème linéaire :

Soit $v \in H^1(\Omega)$,

$$\begin{aligned} \int_{\Omega} \operatorname{div}(\rho_{n-1} \nabla \phi_n) v &= - \int_{\Omega} \rho_{n-1} \nabla \phi_n \cdot \nabla v + \int_{\partial\Omega} \rho_{n-1} \partial_N \phi_n v + \int_{\Omega_{ext}} \rho_{n-1} \partial_N \phi_n v \\ &= - \int_{\Omega} \rho_{n-1} \nabla \phi_n \cdot \nabla v + \int_{\Omega_{ext}} \rho_{n-1} v u_\infty \\ &= 0 \end{aligned}$$

La formulation variationnelle s'écrit alors :

$$\int_{\Omega} \rho_{n-1} \nabla \phi_n \cdot \nabla v = \int_{\Omega_{ext}} \rho_{n-1} v u_\infty \quad \forall v \in H^1(\Omega)$$

Pour un écoulement, le nombre de mach est donné par $M = \frac{u_\infty}{c^\star}$. Dans le cadre de l'écoulement considéré, nous avons $M = u_\infty$. Dans un premier temps nous choisissons $u_\infty = 0.3$

On implémente cette méthode avec FreeFem++, ce qui nous mène au code suivant :

```

1 real cEtoile = 1.0;
2 real mach = 0.3;
3 real Uinf = mach * cEtoile;
4 real eps = 1e-4;
5 real erreur = 1.0;
```

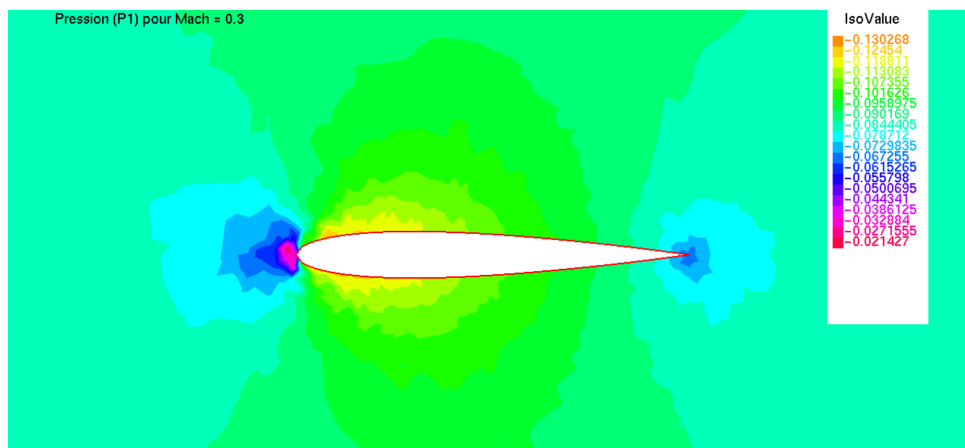
```

6  int iterMax = 100;
7
8  fespace Vh(Th, P1);
9  Vh phi, v;
10 Vh rho = 1.0;
11 Vh rhoOld = 0.0;
12
13
14 problem Ecoulement(phi, v, solver=LU) =
15   int2d(Th)( rho * (dx(phi)*dx(v) + dy(phi)*dy(v)) + 1e-6*phi*v )
16   - int1d(Th, 2)( rho * Uinf * N.x * v );
17
18 int iter = 0;
19
20 while(erreur > eps && iter < iterMax) {
21
22   iter++;
23   Ecoulement;
24   rhoOld = rho;
25
26   rho = max(1e-5, 1.0 - (1.0/6.0)*(dx(phi)^2 + dy(phi)^2))^( -2./7.);
27
28   Vh diff = rho - rhoOld;
29   erreur = diff[].linf;
30 }
31
32 Vh pression = -(dx(phi)^2 + dy(phi)^2);

```

Listing 4 – Résolution par point fixe et éléments finis $P1$

Après résolution nous obtenons cette répartition de pression de la pression autour de l'aile :

FIGURE 28 – Champ de pression autour de l'aile, éléments finis $P1$

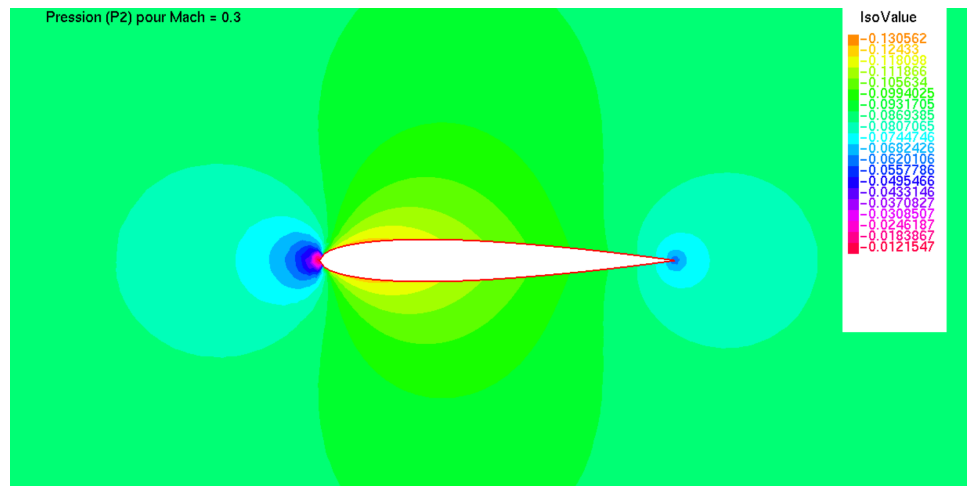


FIGURE 29 – Champ de pression autour de l’aile, éléments finis $P2$

Question 2

On fait maintenant varier le nombre de mach à 0.5 :

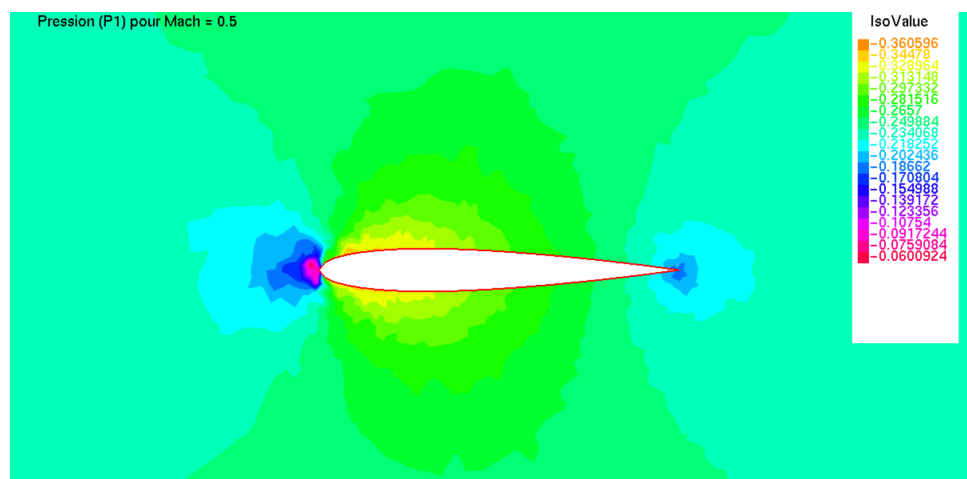


FIGURE 30 – Champ de pression autour de l’aile, éléments finis $P1$, $M = 0.5$

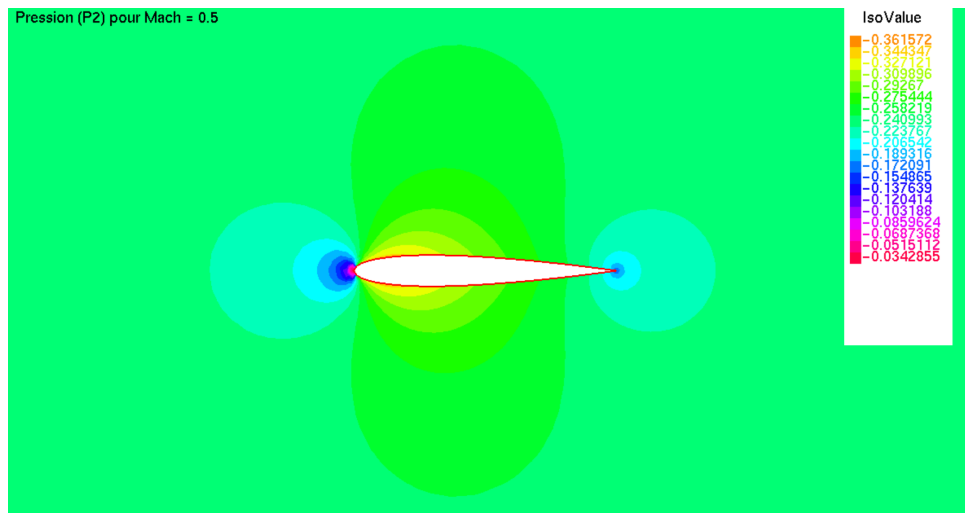


FIGURE 31 – Champ de pression autour de l’aile, éléments finis P_2 , $M = 0.5$

Puis maintenant nous prenons le nombre de mach égal à 0.7.

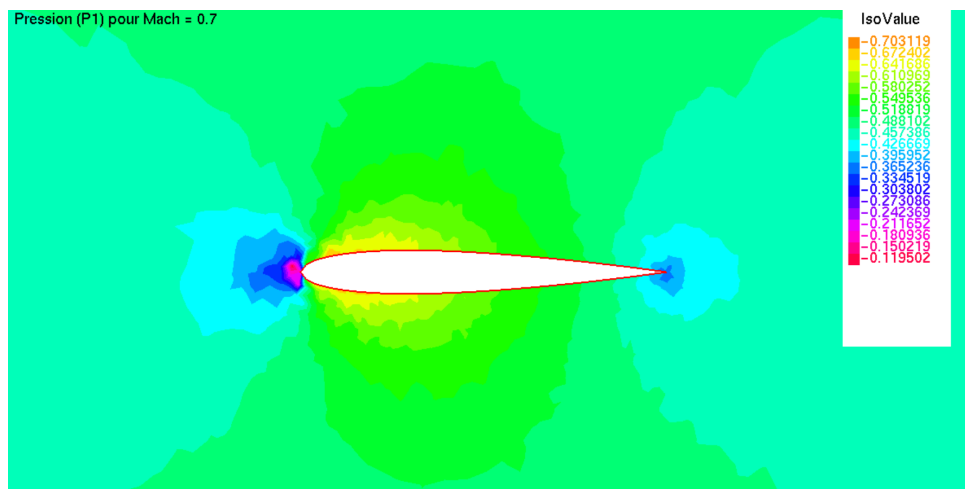


FIGURE 32 – Champ de pression autour de l’aile, éléments finis P_1 , $M = 0.7$

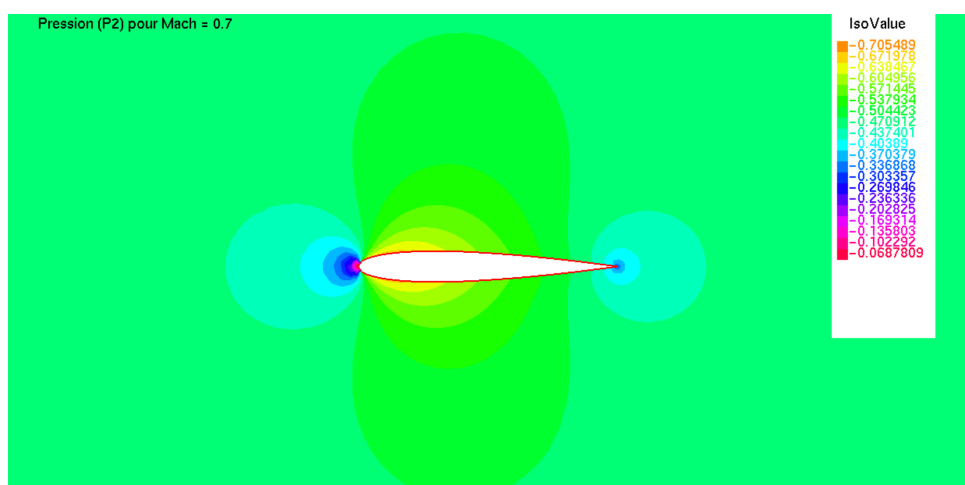


FIGURE 33 – Champ de pression autour de l’aile, éléments finis P_2 , $M = 0.7$